SYSTEM REQUIREMENT SPECIFICATION (SRS)

AI Web Scraping & Intelligent Content Summarizer

Major Project | B.Tech. Computer Science & Engineering

Submitted by

Gaurav Chauhan (1230432288) | Gaurav Yadav (1230432292)

Golu Rai (1230432295) | Deepal Yadav (1230432259)

Under the supervision of

Dr. Harsh Dev (Head of Department)

Department of Computer Science & Engineering

School of Engineering

BABU BANARASI DAS UNIVERSITY, LUCKNOW

Document Information

Field	Details
Document Title	System Requirement Specification (SRS)
Project Title	AI Web Scraping & Intelligent Content Summarizer
Document Type	Major Project Requirement Gathering / SRS
Institution	Babu Banarasi Das University, Lucknow
Department	Computer Science & Engineering
Prepared From	SRS Sample Documentation and Revised Project Synopsis
Version	1.0

Note: The sample SRS provides the document structure. All system features and project boundaries below are taken from the supplied synopsis. Where the synopsis does not define a detail (for example, account roles beyond the user account, exact file-size limits, or numerical response-time targets), it is identified as unspecified rather than invented.

1. Executive Summary & Project Objectives

AI Web Scraping & Intelligent Content Summarizer is a web-based platform that collects content from uploaded PDF, DOCX and TXT files, public web pages, and YouTube videos that expose transcripts. It extracts text and source metadata, preprocesses and segments content into meaningful sections and clauses, classifies clauses with a project-trained transformer model, identifies items that deserve closer reading, generates multi-level summaries, supports natural-language questions through semantic retrieval and Retrieval-Augmented Generation (RAG), and compares two document versions to report meaningful changes.

The system is intended to help users understand lengthy and complex documents, especially terms of service, privacy policies, rental agreements, offer letters and service contracts. It is an informational comprehension aid, not a legal-advice system and does not decide whether a clause is lawful, enforceable or fair.

1.1 Project Objectives

- Prepare a labelled clause dataset through collection, cleaning, segmentation, manual labelling, deduplication and validation.
- Fine-tune a pretrained BERT-family transformer encoder for supervised multi-class clause classification and return a category with a confidence score.
- Evaluate the classifier using accuracy, precision, recall, F1-score (macro and weighted), confusion matrix and class-wise error analysis.
- Ingest and extract text and metadata from PDF, DOCX, TXT, web pages and YouTube transcripts.
- Clean and normalize text, identify sections, segment sentences and clauses, and preserve page, section and timestamp references.
- Enable semantic search using Sentence Transformer embeddings and FAISS, and answer user questions through a grounded RAG pipeline with source references.
- Assign a Low, Medium, High or Very High reading-priority indicator using predicted category, confidence and content signals.
- Generate overall, section-wise, important-clause and question-focused summaries.
- Compare two document versions and report added, removed and modified clauses, with emphasis on fees, renewal, cancellation and data use.
- Provide a secure React web interface connected to a FastAPI backend, database, dashboard, document history and analysis views.

1.2 Primary Users and Target Platform

User / Component	Requirements
End User	Uses the web interface to upload a file or submit a URL/video link, review analysis and summaries, ask questions, inspect clause labels and attention levels, and compare documents.
Web Application	React.js interface with Tailwind CSS; responsive dashboard

	and visualizations using Recharts where appropriate.
Backend Service	Python with FastAPI REST APIs for authentication, ingestion, analysis, summaries, questions and document comparison.
ML / NLP Pipeline	Pretrained transformer fine-tuned on a project-labelled clause dataset, NLP processing, semantic embeddings, FAISS retrieval and RAG.
Database	PostgreSQL accessed through SQLAlchemy for users, documents, chunks, clauses, conversations, messages and model-version metadata.

2. User Roles, Onboarding & Access Control

2.1 End User

- The system shall provide account registration and login using an email address and password.
- The system shall issue and verify JWT-based session tokens.
- The system shall restrict access to documents and analysis records to the authenticated owner.
- The user shall be able to upload supported documents or provide a public webpage URL or YouTube link.
- The user shall be able to view document history, access stored analysis, ask document-related questions, and delete a document.

2.2 System / Administrator Functions

The synopsis specifies user authentication and ownership checks but does not define a separate administrator role, administrator dashboard, or multi-role approval workflow. These are not treated as required features in this SRS.

3. Detailed Feature Specifications

3.1 Web Application and Source Submission

- The interface shall allow users to upload PDF, DOCX and TXT files.
- The interface shall allow users to submit a public webpage URL.
- The interface shall allow users to submit a YouTube video link when transcript text is available.
- The application shall identify the submitted source type, validate it and queue it for processing.
- The application shall show the document or source processing status and surface an understandable error when content cannot be processed.
- The dashboard shall provide access to summaries, clause analysis, question answering, attention indicators, visualizations and document comparison.

3.2 Content Extraction and NLP Preprocessing

- The system shall extract text from PDF files using a PDF extraction library and preserve page numbers.
- The system shall extract text from DOCX and TXT files.
- The system shall extract main content from accessible public webpages and preserve available section information.
- The system shall retrieve available YouTube transcript text and preserve timestamps where available.
- The system shall clean and normalize extracted content, remove unnecessary formatting and handle textual inconsistencies.
- The system shall perform sentence segmentation and section identification.
- The system shall segment text into meaningful clauses or content chunks while retaining source metadata required for traceability.
- The system shall detect scanned PDFs without an extractable text layer and report the limitation instead of treating them as correctly processed.

- The system shall report when a webpage blocks automated access or a video has no accessible transcript.

3.3 Clause Dataset Management and Model Training

- The project shall maintain a labelled clause dataset with documented categories and annotation guidance.
- The dataset workflow shall support importing clauses, manual labelling, deduplication, label validation, versioning and stratified train/validation/test splits.
- The training workflow shall tokenize clauses and fine-tune a pretrained BERT-family transformer encoder using a supervised learning approach.
- The training workflow shall support validation, checkpoint selection, class-weighted loss and early stopping as described in the synopsis.
- The system shall record model identifier, training date, dataset version and evaluation metrics for each model version.
- The trained model shall return a predicted clause category and confidence score for incoming clauses.

Clause categories named in the synopsis include financial, automatic renewal, cancellation, privacy, liability, arbitration and termination. Clauses that do not fit a defined category may be assigned to “Other”. The final category set is to be confirmed and documented during dataset preparation.

3.4 Attention / Reading-Priority Analysis

- The system shall calculate an attention score using classifier output, confidence, extracted entities and relevant content signals.
- The system shall assign one of four reading-priority levels: Low, Medium, High or Very High.
- The system shall display an explanation of the factors that contributed to the assigned level.
- The indicator shall represent reading priority only and shall not be presented as a legal-risk score or legal determination.

3.5 Semantic Search and RAG Question Answering

- The system shall generate dense semantic embeddings for document chunks and user questions using a Sentence Transformer.
- The system shall index chunk embeddings in FAISS and retrieve passages by semantic similarity rather than exact keyword matching alone.
- The RAG pipeline shall retrieve relevant chunks, optionally rerank them, assemble context and provide the context with the user question to a configurable language model.
- The system shall generate natural-language answers grounded in retrieved document content.
- Answers shall include source references such as page number, section name or video timestamp when available.
- The system shall retain question-and-answer history associated with the relevant document, subject to authenticated ownership controls.
- The system shall communicate that generated explanations can contain errors and that references should be checked against the original source.

3.6 Intelligent Summarization

- The system shall generate an overall summary for a processed document.
- The system shall generate section-wise summaries.
- The system shall provide summaries of important clauses, including the corresponding attention level and source reference where available.
- The system shall support question-focused summaries based on a user query and retrieved source content.
- The system shall use a hierarchical or chunk-based summarization strategy for documents longer than the model context window.
- Summaries should preserve consequential details such as amounts, deadlines, conditions, renewals, cancellation terms and data-use permissions when present in the source.

3.7 Document Version Comparison

- The system shall accept two versions of a document for comparison.
- The system shall align semantically corresponding clauses across the versions.
- The result shall identify added, removed and modified clauses.
- The comparison view shall draw attention to changes related to fees, automatic renewal, cancellation and privacy/data use.
- The system shall show both versions or the supporting excerpts so that users can verify changes; comparison output shall not be represented as guaranteed complete.

3.8 Dashboard, History and Visualization

- The dashboard shall display clause records with predicted category, confidence, attention level and source location.
- The dashboard shall provide filters and visual summaries of clause categories and attention-level distributions.
- The dashboard shall provide access to question answering and document-comparison views.
- The system shall store document metadata, extracted chunks, clause results, conversations, messages and model-version information.
- The user shall be able to revisit prior documents and their analysis, subject to owner access checks.
- The user shall be able to delete a document; associated extracted text, clauses and vectors shall also be removed as specified by the synopsis.

4. Non-Functional & Technical Requirements

4.1 System Architecture and Technology Stack

Layer	Selected / Proposed Technology	Purpose
Frontend	React.js, Tailwind CSS, Recharts	Responsive web UI, dashboard, filters and visualizations.
Backend	Python, FastAPI, Uvicorn	REST APIs for uploads, URLs, analysis, summarization, questions and comparison.
ML model	PyTorch, Hugging Face Transformers	Fine-tune and serve the supervised transformer-based clause classifier.
NLP / data processing	spaCy or NLTK; Pandas; NumPy; scikit-learn	Preprocessing, dataset handling and model evaluation.
Document extraction	PyMuPDF, python-docx, Trafilatura or BeautifulSoup, YouTube Transcript API	Read supported document formats, accessible webpages and available transcripts.
Embeddings / vector search	Sentence Transformers, FAISS	Semantic chunk embeddings and similarity search.
RAG / generation	Custom RAG pipeline; LangChain where useful; configurable hosted or local LLM	Grounded question answering, explanations and summaries.
Database	PostgreSQL, SQLAlchemy, Alembic	Persistent structured data and schema management.
Authentication / security	JWT, bcrypt or Argon2	Session tokens and password hashing.
Development / testing / deployment	VS Code, Git, GitHub, Postman, Pytest; Vercel and Render/Railway or similar	Version control, API testing, automated tests and hosting.

The synopsis identifies these technologies as the proposed stack and allows the final language-model provider and some extraction libraries to be selected based on compatibility, resource availability and implementation needs.

4.2 Security and Privacy

- Passwords shall be hashed using bcrypt or Argon2; plaintext passwords shall not be stored.
- JWT session tokens shall have a limited expiry and be verified on protected requests.
- Every database operation involving user-owned records shall apply authenticated-owner checks.

- Uploaded files shall be validated by extension and content type, size-limited, and stored under generated filenames rather than user-controlled storage paths.
- User-provided URLs shall be validated to reduce the risk of requests to internal network addresses.
- Provider API keys shall be stored in server-side environment variables and shall not be exposed to the frontend.
- API request bodies shall be validated through typed schemas such as Pydantic.
- Error messages shown to users shall be generic where needed, while diagnostic details are recorded on the server.
- Logs shall not include document contents, authentication tokens or personal data.
- The interface shall disclose that content may be transmitted to the configured language-model provider during question answering.
- Deleting a document shall remove its stored text, clauses and associated vectors as specified by the synopsis.

4.3 Reliability, Traceability and Model Evaluation

- The system shall preserve page, section and timestamp metadata wherever available so outputs can be checked against their source.
- The classifier shall be evaluated on held-out test data using accuracy, precision, recall, macro F1, weighted F1 and a confusion matrix.
- The project shall report class-wise behaviour and error analysis; no performance values shall be stated until experiments have been run.
- The system shall provide clear processing errors for unsupported or inaccessible sources rather than silently fabricating content.
- The application shall preserve both document versions in the comparison view because semantic alignment may miss some changes.

4.4 Performance and Resource Requirements

Area	Requirement from Synopsis
Development machine	Intel Core i5 or equivalent; minimum 8 GB RAM, 16 GB recommended; 100 GB free disk space.
Model fine-tuning	CUDA-capable GPU with at least 8 GB memory, or a cloud GPU environment such as Google Colab or Kaggle.
Deployment environment	Cloud instance with at least 2 vCPU and 4 GB RAM for CPU-based inference, with a stable internet connection.
Response-time targets	Specific numeric latency or throughput targets are not stated in the synopsis and require confirmation.
Scalability target	A numerical concurrent-user or maximum-document-size target is not specified in the supplied synopsis.

4.5 Hardware and Software Requirements

Type	Specification
Operating System	Windows 10/11, Ubuntu 20.04 or later, or macOS.
Languages	Python 3.10 or later; JavaScript (ES6+).
ML / NLP libraries	PyTorch, Hugging Face Transformers, Sentence Transformers, scikit-learn, spaCy, NLTK, Pandas and NumPy.
Vector search / extraction	FAISS; PyMuPDF; python-docx; Trafilatura; BeautifulSoup; YouTube Transcript API.
Web application	FastAPI with Uvicorn; React.js with Tailwind CSS and Recharts; Node.js and npm.
Database	PostgreSQL with SQLAlchemy and Alembic.
Tools	VS Code, Git/GitHub, Postman, Jupyter Notebook and Pytest.
Deployment	Vercel for frontend and Render, Railway or a similar service for backend, subject to available resources.

5. System Workflow

1. User submits an uploaded file, webpage URL or YouTube link through the React web application.
2. The ingestion layer validates the input, detects the source type and selects an extraction method.
3. The extraction module obtains readable text and preserves page, section or timestamp metadata.
4. The NLP pipeline cleans and normalizes text, identifies sections, segments sentences and clauses, and forms metadata-aware chunks.
5. The trained clause classifier assigns category and confidence values; the attention module determines a four-level reading priority.
6. The embedding module creates vectors and stores/searches them through FAISS.
7. For a question, the RAG pipeline retrieves relevant passages and sends grounded context to the configured language model.
8. The summarization and comparison modules prepare overall/section/clause/question-focused summaries and version-change reports.
9. The dashboard presents summaries, clause classifications, attention explanations, source references, search answers and comparison results.

Workflow representation: User → React Web Application → Source Ingestion → Content Extraction → NLP Preprocessing / Clause Segmentation → ML Classification & Attention Analysis → Embeddings / FAISS → RAG + LLM → Summaries / Comparison / Source References → Dashboard.

6. Data Requirements and Database Entities

Entity	Purpose and Key Attributes
Users	Account identity, email, hashed password and registration timestamp.
Documents	Owner, title, source type, original reference, checksum and processing status.
Chunks	Parent document, chunk text, page and section metadata, and embedding reference.
Clauses	Parent document, clause text, predicted category, confidence, attention level and source location.
Conversations	Question-answering session associated with a document.
Messages	User questions, generated answers and the references used.
Model Versions	Model identifier, training date, dataset version and evaluation metrics.

7. API Requirements

The synopsis proposes a FastAPI REST backend and lists the following representative endpoints. Exact request/response schemas and status codes should be defined during implementation.

Method	Endpoint	Purpose
POST	/api/auth/register	Register a user account.
POST	/api/auth/login	Authenticate a user.
POST	/api/documents/upload	Upload a PDF, DOCX or TXT document.
POST	/api/documents/url	Submit a webpage URL.
POST	/api/documents/youtube	Submit a YouTube video link.
GET	/api/documents	List the authenticated user's documents.
GET	/api/documents/{id}	Retrieve document metadata and processing information.
POST	/api/documents/{id}/ask	Ask a question about a document using RAG.
POST	/api/documents/{id}/summarize	Generate a document summary.
GET	/api/documents/{id}/clauses	Retrieve classified clauses and their metadata.
GET	/api/documents/{id}/risk-analysis	Retrieve attention / reading-priority analysis.

POST	/api/documents/compare	Compare two document versions.

8. Scope of the Project

8.1 Included in Scope

- PDF, DOCX and TXT ingestion; accessible public webpages; YouTube videos with available transcripts.
- Text extraction with available page, section and timestamp metadata.
- NLP cleaning, normalization, section identification, sentence segmentation and clause segmentation.
- Labelled dataset preparation and fine-tuning/evaluation of a task-specific transformer clause classifier.
- Sentence embeddings, FAISS semantic search and RAG question answering with source references.
- Attention/reading-priority levels, hierarchical multi-level summarization and semantic document comparison.
- Authenticated React web application, FastAPI backend, PostgreSQL database, document history, dashboard and testing.

8.2 Excluded from Scope

- Training a large language model from scratch; the project fine-tunes a pretrained encoder and uses an existing generative model for explanations and question answering.
- Legal decision-making or determinations about enforceability, legality or fairness of clauses.
- Executing actions for users, such as cancelling subscriptions or taking legal/financial action on their behalf.
- Optical character recognition for scanned documents in the first version.
- Languages other than English in the first version.

8.3 Assumptions and Constraints

- Uploaded documents are assumed to contain machine-readable text.
- Webpages are assumed to permit automated extraction of their main content.
- YouTube sources are assumed to expose transcript text.
- Classifier quality depends on the diversity, quality and size of the labelled dataset.
- Fine-tuning is constrained by available compute; a base-sized transformer and free-tier GPU environments may be used.
- Hosted LLM latency, cost and usage limits are outside the project team's direct control.
- Webpages rendered only through unsupported client-side scripts, pages restricting automated access, and videos without transcripts may not be processed reliably.

9. Acceptance Criteria

The following criteria translate the synopsis into observable project checks. They do not impose numerical accuracy or latency claims that have not yet been measured.

ID	Acceptance Criterion
AC-01	A user can register, log in and access only their own stored documents.
AC-02	The application accepts PDF, DOCX and TXT files and attempts extraction while preserving available source metadata.
AC-03	The application accepts a public webpage URL and a YouTube link with an accessible transcript, and reports unsupported/inaccessible sources clearly.
AC-04	NLP processing produces normalized text and meaningful sections/clauses while preserving traceability information.
AC-05	The trained classifier produces a clause category and confidence score; evaluation reports accuracy, precision, recall, macro/weighted F1 and a confusion matrix.
AC-06	Clause analysis assigns Low, Medium, High or Very High

	reading priority and explains relevant factors.
AC-07	Semantic search retrieves relevant passages and RAG answers include available page/section/timestamp references.
AC-08	The system produces overall, section-wise, important-clause and question-focused summaries.
AC-09	Comparison reports added, removed and modified clauses and lets users inspect source versions/excerpts.
AC-10	Users can view document history and delete documents with associated extracted text, clauses and vectors.
AC-11	Automated and integration tests cover extraction, preprocessing, scoring and the end-to-end pipeline.
AC-12	The UI states the system is an informational aid and does not provide legal advice or guarantee completeness.

10. Testing and Evaluation Requirements

- Unit tests shall cover source extraction, NLP preprocessing, clause segmentation and attention scoring.
- Integration tests shall exercise the ingestion-to-dashboard pipeline, including retrieval, question answering and summarization.
- Authentication and ownership checks shall be tested to prevent cross-user document access.
- File validation, malformed inputs, inaccessible URLs, scanned PDFs and transcript-unavailable video cases shall be tested.
- Classifier performance shall be measured on held-out test data and summarized with the metrics defined in Section 4.3.
- Document comparison tests shall include additions, removals and modifications, while recording that semantic alignment may not identify every change.
- The final report shall include the actual measured model results and known limitations; values shall not be fabricated before experiments are complete.

11. Summary Matrix of Decisions

Parameter	Selected Logic / Rule
Project Platform	Web-based application: React.js frontend and FastAPI backend.
Source Types	PDF, DOCX, TXT, public webpage URL and YouTube transcript source.
Language	English in the first version.
Text Pipeline	Extraction → cleaning / normalization → section and clause segmentation with source metadata.
Classifier	Pretrained BERT-family transformer fine-tuned on the project-labelled clause dataset.
Classification Output	Clause category and confidence score; “Other” for unmatched categories where applicable.
Attention Levels	Low, Medium, High and Very High; an indicator of reading priority, not legal risk.
Semantic Retrieval	Sentence Transformer embeddings indexed and searched using FAISS.
Question Answering	RAG with retrieved context and source/page/section/timestamp references.
Summarization	Overall, section-wise, important-clause and question-focused summaries.
Comparison	Added, removed and modified clauses across two document versions.
Database	PostgreSQL with SQLAlchemy; stored vectors in FAISS with linking identifiers in the database.
Security	JWT authentication, hashed passwords, input/file validation, per-user access checks and server-side API keys.
Known Exclusions	No OCR for scanned documents in v1; no legal determinations or actions on the user’s behalf.
Evaluation	Accuracy, precision, recall, macro/weighted F1, confusion

	matrix and class-wise error analysis.

12. Expected Outcomes

- A trained clause-classification model that returns categories and confidence scores for unseen clauses.
- Semantic retrieval and grounded question answering based on a user's own source material.
- A four-level reading-priority indicator with an explanation of contributing factors.
- Plain-English summaries and explanations with verifiable source references.
- A multi-source pipeline that ingests supported documents, webpages and transcript-enabled video content.
- A comparison view identifying added, removed and modified clauses across two versions.
- A secure web dashboard with authentication, document history, visualizations and test documentation.

13. Project Limitations and Risks

- Classifier performance may be lower for clause styles or document types missing from the training data.
- The defined label set cannot cover every possible clause; unmatched content may be classified as "Other".
- Scanned PDFs are not processed through OCR in the first version.
- Restricted or dynamically rendered webpages and videos without available transcripts may fail extraction.
- Generated answers and summaries can contain errors; source references are provided to support verification.
- The attention indicator is a reading-priority aid, not a legal-risk assessment.
- Externally hosted language models may add latency, cost and service limits.
- The first version supports English only, and semantic document comparison cannot be guaranteed complete.

14. Work Plan / Timeline

Month	Planned Activities	Deliverable
1	Literature review, requirements analysis, finalise clause categories, evaluate extraction libraries and set up the project.	Requirement specification and annotation guideline.
2	Multi-source ingestion and extraction, NLP preprocessing, clause segmentation and metadata handling.	Ingestion and segmentation pipeline.
3	Dataset collection, cleaning, clause extraction, manual labelling, deduplication, validation and stratified split.	Labelled and validated clause dataset.
4	Fine-tuning experiments, hyperparameter tuning, validation, early stopping and model selection.	Trained clause-classification model.
5	Full evaluation, confusion matrix, class-wise analysis, error analysis and backend integration.	Evaluation report and deployed classifier.
6	Embedding generation, FAISS indexing, retrieval tuning, RAG, source references and summarization.	Semantic search and question answering.
7	Attention scoring, document comparison, React dashboard, visualizations, authentication and database integration.	Integrated web application.
8	Unit/integration testing, performance tuning, security review, deployment and final documentation/presentation.	Deployed system and final report.

15. Requirement Traceability to Synopsis

SRS Section	Synopsis Basis
Executive Summary / Objectives	Synopsis Sections 1–3: Introduction, Problem Statement and Objectives.
Functional Requirements	Synopsis Sections 4, 6 and 7: Scope, Methodology / Proposed System and Module Description.
Technology Stack	Synopsis Section 6.2: Tools and Technologies.
Database / APIs / Security	Synopsis Sections 6.3–6.5: Database Design, Backend API Design, Security and Privacy.
Acceptance / Testing	Synopsis Sections 3, 7, 8 and 11: Objectives, Module Description, Expected Outcomes and Work Plan.
Scope / Limitations	Synopsis Sections 4 and 9: Scope and Project Limitations.
Hardware / Software / Timeline	Synopsis Sections 10 and 11: Hardware and Software Requirements, Work Plan / Timeline.
Format / SRS organisation	SRS Sample Documentation: project summary, user roles, detailed features, non-functional/technical requirements and summary matrix.